In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

1. GENERATE RAW MOCK DATA

In [2]:
```
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],  # Missing value
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28",
    ],
}

df = pd.DataFrame(raw_data)

print("--- Raw Data ---")
print(df)
print()
```

```
--- Raw Data ---
  Employee_Name   Department   Salary   Join_Date
0         Alice           HR  50000.0  2022-01-15
1           Bob  Engineering  85000.0  2021-06-20
2       Charlie           HR      NaN  2023-03-11
3         Diana  Engineering  92000.0  2020-11-01
4          Evan    Marketing  60000.0  2024-02-28
```

2. DATA PREPROCESSING

In [3]:
```
# Clean whitespace from employee names
df["Employee_Name"] = df["Employee_Name"].str.strip()

# Fill missing salary with median salary
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

# Convert Join_Date to datetime format
df["Join_Date"] = pd.to_datetime(df["Join_Date"])

# Create Years_of_Service column (Current Year = 2026)
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("--- Preprocessed Data ---")
print(df)
print()
```

```
--- Preprocessed Data ---
  Employee_Name   Department   Salary  Join_Date  Years_of_Service
0         Alice           HR  50000.0 2022-01-15                 4
1           Bob  Engineering  85000.0 2021-06-20                 5
2       Charlie           HR  72500.0 2023-03-11                 3
3         Diana  Engineering  92000.0 2020-11-01                 6
4          Evan    Marketing  60000.0 2024-02-28                 2
```

3. DATA AGGREGATION

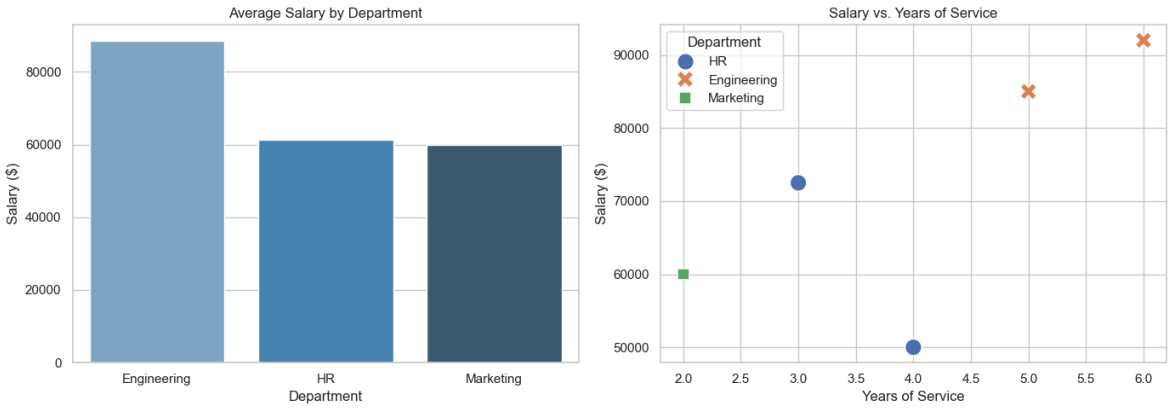
In [4]:
```
dept_summary = (
    df.groupby("Department")
      .agg(
          Avg_Salary=("Salary", "mean"),
          Total_Employees=("Employee_Name", "count")
      )
      .reset_index()
)

print("--- Aggregated Summary ---")
print(dept_summary)
print()
```

```
--- Aggregated Summary ---
    Department  Avg_Salary  Total_Employees
0  Engineering     88500.0                2
1           HR     61250.0                2
2    Marketing     60000.0                1
```

4. DATA VISUALIZATION

In [5]:
```
# Set visual theme
sns.set_theme(style="whitegrid")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar Chart
sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    palette="Blues_d",
    legend=False,
    ax=axes[0]
)

axes[0].set_title("Average Salary by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Salary ($)")

# Scatter Plot
sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
```

```
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1]
)

axes[1].set_title("Salary vs. Years of Service")
axes[1].set_xlabel("Years of Service")
axes[1].set_ylabel("Salary ($)")

plt.tight_layout()
plt.show()
```



1. Creating Sample DataFrame

In [6]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame()

df["Name"] = ["John", "Emma", "Liam", "Olivia"]
df["Age"] = [20, 19, 21, 18]
df["Student"] = [True, True, False, True]

print(df)
```

```
    Name  Age  Student
0   John   20     True
1   Emma   19     True
2   Liam   21    False
3  Olivia  18     True
```

2. Adding a Row to DataFrame

In [7]:
```
import pandas as pd

# Existing DataFrame
df = pd.DataFrame({
    "Name": ["John", "Emma", "Liam", "Olivia"],
    "Age": [20, 19, 21, 18],
    "Student": [True, True, False, True]
})

# New Row
new_row = pd.DataFrame(
```

```
    [["Sophia", 22, False]],
    columns=["Name", "Age", "Student"]
)

# Add row
df = pd.concat([df, new_row], ignore_index=True)

print(df)
```

```
     Name  Age  Student
0    John   20     True
1    Emma   19     True
2    Liam   21    False
3  Olivia   18     True
4  Sophia   22    False
```

3. Measures of Central Tendency (SciPy)

In [8]:
```
from scipy import stats
import numpy as np

data = [10, 20, 30, 40, 50]

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))
```

```
Mean: 30.0
Median: 30.0
Mode: ModeResult(mode=np.int64(10), count=np.int64(1))
```

4. Probability Distribution Analysis

In [9]:
```
from scipy.stats import norm

probability = norm.cdf(85, loc=70, scale=10)

print("Probability:", probability)
```

```
Probability: 0.9331927987311419
```

5. Hypothesis Testing

In [10]:
```
from scipy import stats

data = [22, 25, 19, 24, 28, 30]

t_stat, p_value = stats.ttest_1samp(data, 25)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

```
T-Statistic: -0.2049800154226962
P-Value: 0.8456711339658732
```

6. Correlation Analysis

In [11]:
```
from scipy.stats import pearsonr

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

corr, p_value = pearsonr(x, y)

print("Correlation:", corr)
print("P-value:", p_value)
```

```
Correlation: 1.0
P-value: 0.0
```

7. Linear Algebra Operations

In [12]:
```
from scipy import linalg

A = [[3, 2],
     [1, 2]]

B = [5, 5]

solution = linalg.solve(A, B)

print(solution)
```

```
[0.  2.5]
```

8. Optimization Using SciPy

In [13]:
```
from scipy.optimize import minimize

def objective(x):
    return x**2 + 4

result = minimize(objective, x0=5)

print(result.x)
```

```
[-2.62955131e-08]
```

9. Logistic Regression (Scikit-learn)

In [14]:
```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Load Iris Dataset
iris = datasets.load_iris()

X = iris.data
y = iris.target

# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

# Standardize Data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Model
log_reg = LogisticRegression()

log_reg.fit(X_train, y_train)

# Prediction
y_pred = log_reg.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

10. K-Means Clustering

In [16]:

```
import os
import warnings

# Suppress warnings
warnings.filterwarnings("ignore")

# Fix Windows MKL warning
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["LOKY_MAX_CPU_COUNT"] = "4"

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load Iris dataset
iris = load_iris()
```

```
X = iris.data

# Create KMeans model
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

# Train the model
kmeans.fit(X)

# Display cluster labels
print("Cluster Labels:")
print(kmeans.labels_)

# Display cluster centers
print("\nCluster Centers:")
print(kmeans.cluster_centers_)
```

```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 2 2 2
 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 2 0 2 2 2 2 0 2 2 2 0 2 2 2 0 2
 2 0]

Cluster Centers:
[[5.9016129  2.7483871  4.39354839 1.43387097]
 [5.006      3.428      1.462      0.246     ]
 [6.85       3.07368421 5.74210526 2.07105263]]
```

In [ ]: